Report FVAB

Carlo di Vico, Francesca Pellegrino, Carmine Cataldo

Contents

1 Introduzione **2**
- 1.1 Obiettivo . . . 2
- 1.2 Background teorico . . . 2
- 1.3 YOLO . . . 2
- 1.4 Dataset . . . 3
- 1.5 Architettura dell'auto: JetAuto . . . 3
- 1.6 La mente dell'auto: NVIDIA Jetson Nano . . . 3

2 Workflow **5**
- 2.1 Analisi del problema . . . 5
- 2.2 Come abbiamo affrontato il problema? . . . 8

3 Risultati **11**
- 3.1 Dataset finale . . . 11
- 3.2 Metriche da osservare . . . 14
- 3.3 Ottimizzazione per la nostra auto . . . 14
- 3.4 Risultati ottenuti e analisi . . . 14
- 3.5 Modello migliore . . . 15

1 Introduzione

1.1 Obiettivo

Il nostro progetto mira alla costruzione di un modello di detection e riconoscimento di segnali stradali. Il modello dev'essere ottimizzato per la il robot JetAuto con la scheda NVIDIA Jetson Nano. L'obiettivo è migliorare le performance del modello rispetto all'anno precedente in cui l'auto (una Picar X) falliva al momento del testing.

Inoltre, il nostro modello mira a migliorare anche la quantità di segnali riconoscibili. Abbiamo difatti aggiunto i seguenti segnali:

- **Segnale di rotatoria** : segnale con sfondo blu e le classiche frecce bianche che formano un cerchio
- **Segnale di Parcheggio**: segnale con sfondo blu e la P in bianco
- **Segnale bidirezionale avanti o a sinistra**: era già presente il segnale bidirezionale con obbligo di andare avanti o svoltare a destra e abbiamo scelto di aggiungere anche il segnale speculare, il segnale è composto da una doppia freccia, una che punta "dritto" e una che punta a sinistra con una leggera curvatura. Le frecce sono bianche mentre lo sfondo è blu
- **Segnali con obbligo di svolta a destra** e **obbligo di svolta a sinistra**: entrambi i segnali, speculari, sono costituiti da una freccia allungata bianca su sfondo blu, la freccia presenta una curvatura e ha la testa nella direzione in cui è obbligatoria la svolta

Abbiamo dovuto anche ottimizzare i movimenti della macchinina basandoci sulla nuova architettura.

1.2 Background teorico

Un componente chiave dei sistemi di guida autonoma è il cosiddetto Object and Event Detection and Response (OEDR): la capacità del sistema di rilevare oggetti nell'ambiente, come ad esempio veicoli, pedoni, segnali stradali, e di consequenza, rispondere in modo appropriato. L'object detection è oggi affrontata attraverso tre grandi famiglie di approcci: reti convoluzionali (CNN-based, YOLO), architetture Transformer-based (DETR e varianti), e sistemi di fusione multimodale che combinano camera e LiDAR nello spazio Bird's-Eye-View (BEV).

1.3 YOLO

La famiglia YOLO rimane il riferimento principale per la detection in tempo reale. Nei complessi scenari di traffico urbano, segnali stradali piccoli o pedoni lontani possono essere rilevati con maggiore accuratezza grazie a varianti come YOLOv8n o YOLOv8s, con i quali possiamo affrontare il problema dei falsi

allarmi causati da target. Approcci migliorati su YOLOv5 raggiungono accuratezze del 96,1% su dataset come KITTI con velocità di 202 FPS, superiori a molti algoritmi mainstream, grazie all'aggiunta di un layer dedicato agli oggetti piccoli e meccanismi di attenzione coordinata (coordinate attention) su tutti i layer di detection.

Abbiamo usato proprio questa famiglia per addestrare il nostro modello. Nello specifico ci siamo soffermati su addestrare diversi modelli della famiglia YOLO su di un dataset dell'anno scorso provando però a migliorare i risultati ottenuti, nello specifico ci siamo settati l'obiettivo del miglioramento dell'accuracy in fase di testing.

1.4 Dataset

Il dataset con cui abbiamo lavorato era già parzialmente costruito dal team che ha lavorato l'anno precedente al progetto. Abbiamo però trovato diverse problematiche per il dataset che affrontiamo meglio nella sezione successiva.

Il dataset è diviso in due tipi di dati: immagini (in formato .png o .jpg) e label (file .txt che contengono le coordinate dei vari oggetti in formato YOLO con label associate. Per ogni immagine abbiamo un file .txt associato.

1.5 Architettura dell'auto: JetAuto

JetAuto è una piattaforma robotica entry-level creata appositamente per chi vuole studiare e sperimentare con ROS (Robot Operating System). Utilizza la Jetson Nano come unità di elaborazione centrale per gestire tutti i suoi sensori avanzati e muoversi nello spazio in modo intelligente.

Ecco la sua dotazione hardware principale:

- Telecamera di profondità 3D (spesso montata su un supporto motorizzato pan-tilt) per la percezione visiva.
- Sensore Lidar per scansionare l'ambiente a 360 gradi.
- Schermo touch da 7 pollici per il debug e per visualizzare cosa "vede" il robot.
- Motori con encoder per un controllo millimetrico dei movimenti.
- Array di 6 microfoni per l'interazione vocale.

Ovviamente, la parte più interessante per noi sussiste nella telecamera di profondità che permette all'auto di interfacciarsi con l'ambiente esterno.

1.6 La mente dell'auto: NVIDIA Jetson Nano

La Jetson Nano è una scheda di sviluppo creata da NVIDIA che assomiglia molto a un Raspberry Pi, ma con una differenza fondamentale: è pensata per l'Intelligenza Artificiale. A differenza dei normali mini-PC, ha una piccola ma

potente scheda video a 128 core che le permette di eseguire reti neurali e calcoli complessi in tempo reale. Un altro vantaggio è rappresentato dal basso consumo: lavora consumando solo tra i 5 e i 10 Watt. Infine, risulta perfetta per analizzare flussi video, riconoscere oggetti, tracciare volti o leggere la segnaletica stradale.

NVIDIA ha recentemente presentato anche versioni più potenti come la Jetson Orin Nano, progettata per far girare modelli di IA generativa direttamente in locale, ma questa scheda rappresenta un'ottima base di partenza su cui lavorare per far partire il nostro modello.

2 Workflow

2.1 Analisi del problema

La prima cosa che abbiamo fatto è stata analizzare il dataset con cui ha lavorato il precedente gruppo e capire perché falliva nella fase di testing.

Abbiamo dunque deciso di analizzare il dataset e, per farlo, abbiamo scritto del codice (contenuto nel file ReportDataset), che ci genera un report su tutte le informazioni che ci interessano e abbiamo riscontrato diversi problemi:

- **Immagini non pertinenti**: abbiamo trovato diverse immagini che non erano pertinenti con ciò che il modello doveva andare ad analizzare, immagini di volti, segnali sovrapposti l'un l'altro
- **Immagini sporche**: bounding box che coprivano l'intera immagine, quindi spesso molto grandi, soprattutto per alcune classi specifiche (pedoni e semafori vari)
- **Bias centralità**: Quasi la totalità delle immagini ha bounding box che si trovano nella parte centrale dell'immagine. Questo potrebbe essere un chiaro segno di quello che è il problema del progetto precedente: le immagini hanno sempre bounding box centrali, nella visione dell'auto, però, la macchinina non si trova quasi mai con un oggetto perfettamente centrale (ad esempio, un pedone non "compare" all'improvviso al centro ma la macchina deve essere già in allerta o in frenata se vede il pedone, fin dalla parte laterale della visione della camera; stesso per i segnali, nella maggior parte dei casi si trovano sul lato della carreggiata. Se viene posto il segnale sempre al centro, la camera potrebbe non riconoscerlo a pieno)
- **Classi sbilanciate**: Un altro problema del dataset è rappresentato dallo sbilanciamento delle classi, ci sono classi che raggiungono il 22% e altre classi che invece sono a circa il 5%. Una disparità molto forte.

Inoltre, dovendo anche pensare all'aumento di classi abbiamo capito che una fase importante del nostro progetto si concentrava sul lavoro attorno al dataset.

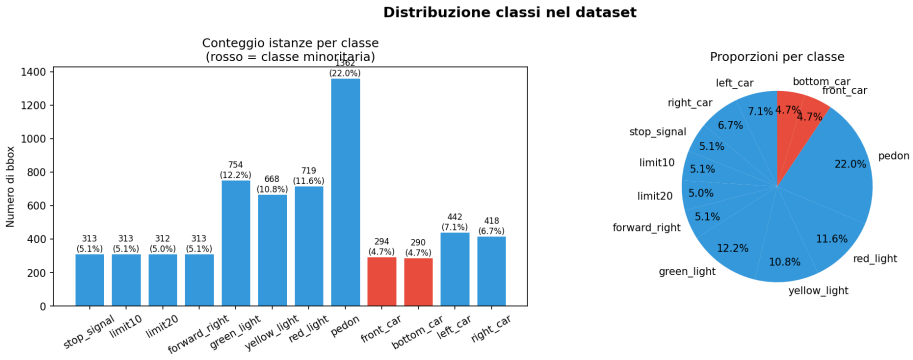


Figure 1: Analisi distribuzione delle classi dataset iniziale

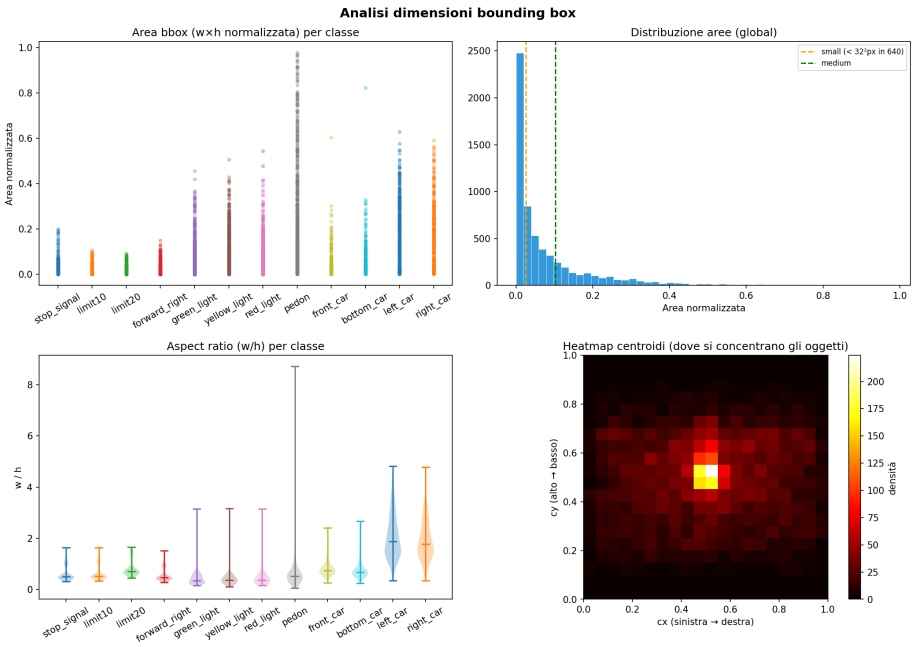


Figure 2: Analisi Bounding Box dataset iniziale

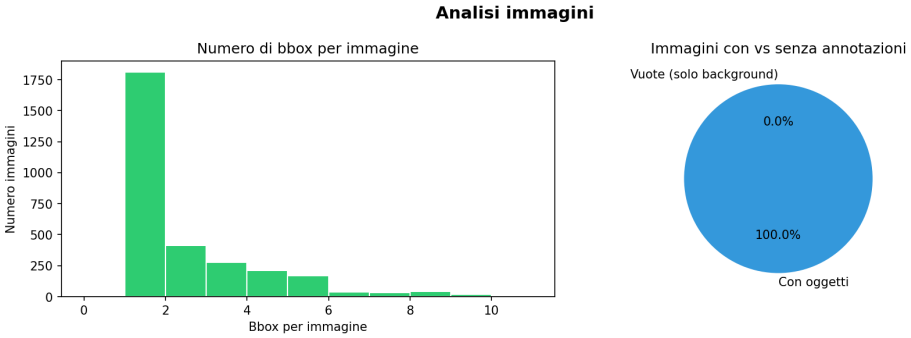


Figure 3: Analisi immagini dataset iniziale

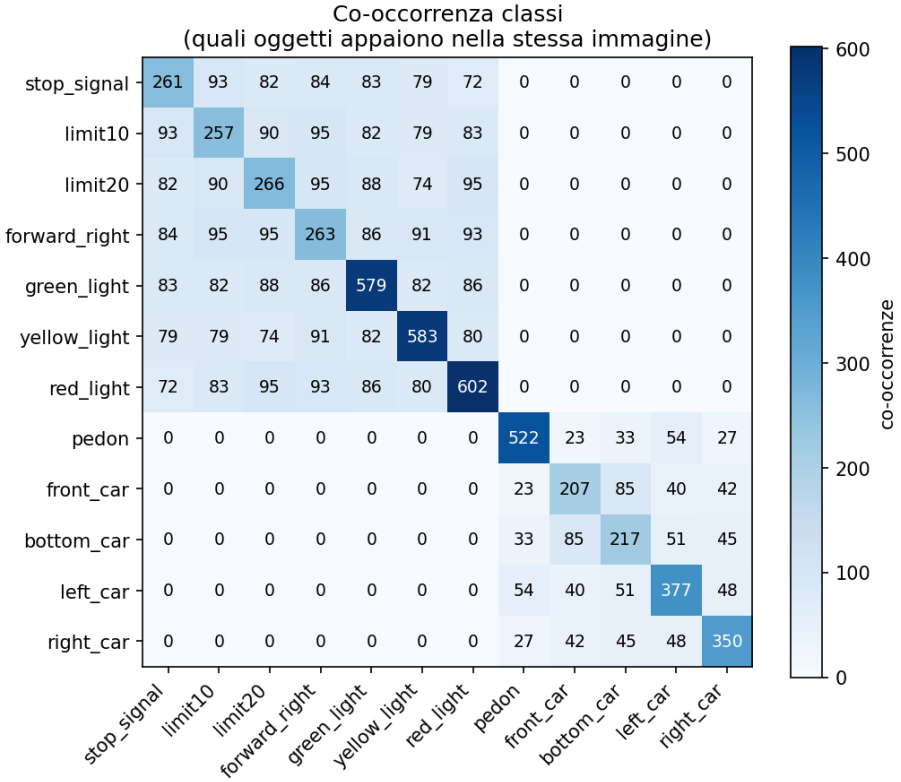


Figure 4: Co-occorrenza tra le varie classi dataset iniziale

2.2 Come abbiamo affrontato il problema?

La prima cosa che abbiamo fatto è stata pulire il dataset da immagini non pertinenti e immagini particolarmente sporche, abbiamo infatti eliminato le immagini sporche o con problemi, eliminando anche le label associate alle immagini, abbiamo inoltre sistemato le varie label e la situazione a questo punto la possiamo vedere nelle seguenti tabelle

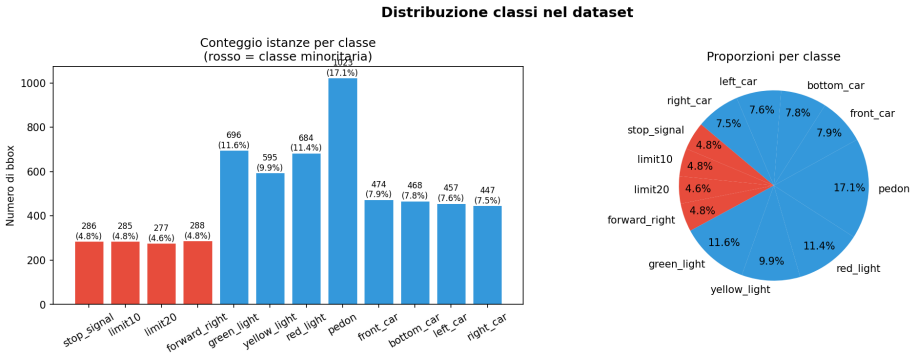


Figure 5: Analisi distribuzione delle classi dataset pulito

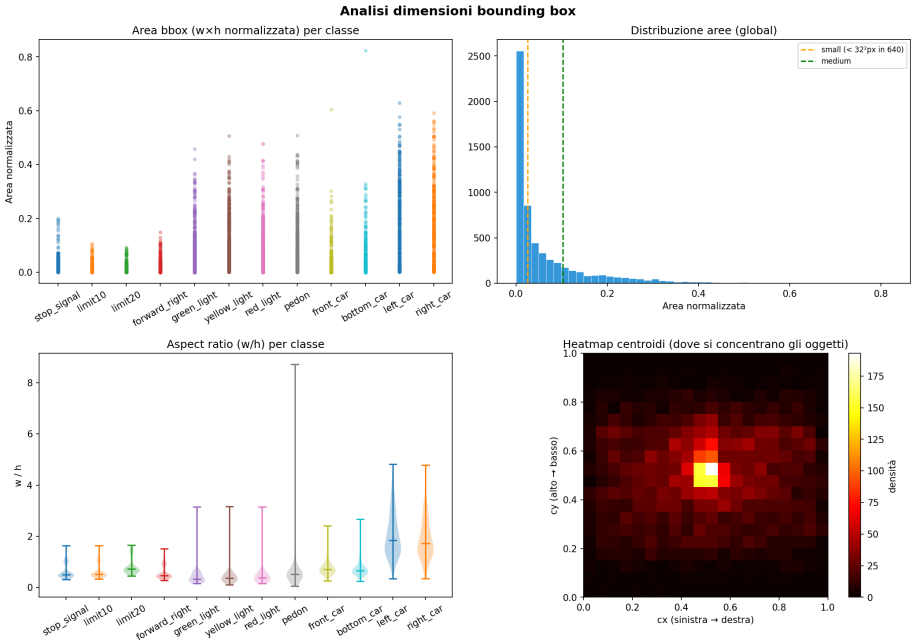


Figure 6: Analisi Bounding Box dataset pulito

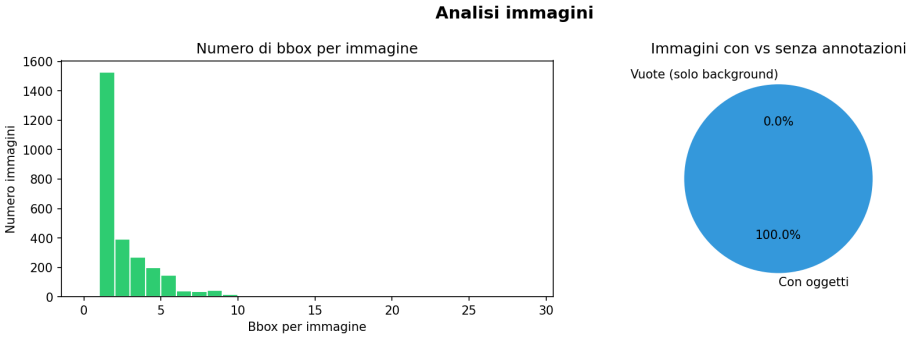


Figure 7: Analisi immagini dataset pulito

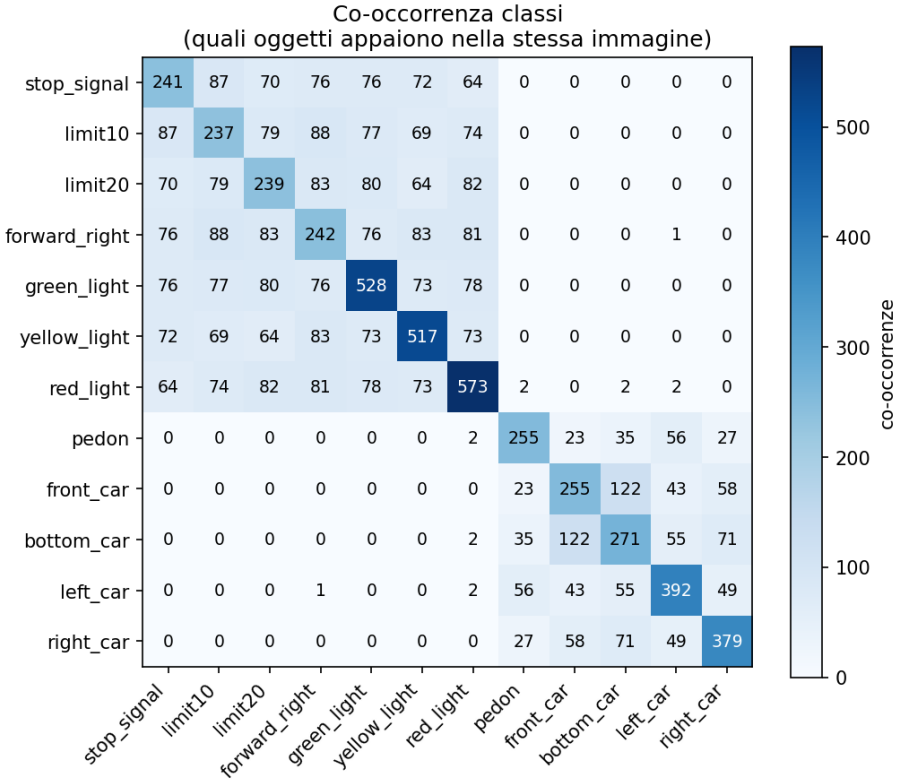


Figure 8: Co-occorrenza tra le varie classi dataset pulito

A questo punto i prossimi passi sono i seguenti:

1. Aggiunta delle nuove classi: abbiamo aggiunto alcune immagini delle

nuove classi, un po' estrapolando etichette nuove dalle immagini che già avevamo a nostra disposizione, un po' aggiungendo al nostro dataset il dataset GTSRB, un dataset che abbiamo trovato in letteratura di segnali stradali che siamo riusciti a portare dal formato "originale" (.ppm) al formato YOLO. Abbiamo poi ripulito le etichette, rimappando le classi con quelle che ci interessano e togliendo le etichette dei segnali non interessanti. Abbiamo quindi lavorato con circa 600 nuove immagini di cui circa 550 vuote. Queste 550 immagini vuote sono state usate successivamente per la data augumentation delle classi nuove.

2. Split del dataset, contenuto nel file split.py, questo script si occupa di prendere il dataset iniziale e dividerlo in due sottoinsiemi: Training Set (80%, nella cartella train) e Validation Set (20%, nella cartella val). Il file prende le immagini e le relative etichette (file .txt) dalla cartella di addestramento e ne sposta casualmente il 20% nelle cartelle di validazione. Dopo aver diviso i dati, lo script analizza i file di testo per contare i bounding box. Genera una tabella e lancia un avviso ("ATTENZIONE SCARLATTA") se rileva che una determinata classe è finita a zero nel Validation Set. Questo è cruciale per evitare che il modello non possa essere validato su determinati oggetti.

3. Augumentation, contenuto nel file augument.py; questo file arricchisce il dataset di addestramento creando nuove immagini artificiali per migliorare la robustezza del modello, senza inquinare i dati di validazione. Crea una copia del dataset per non sovrascrivere i dati originali e poi applichiamo due tipi di augumentazioni:

    - albumentations: Applica trasformazioni visive avanzate (cambi di luminosità/contrasto, sfocatura, traslazioni) alle immagini di training usando la libreria albumentations, aggiornando anche matematicamente le coordinate dei bounding box nel caso delle traslazioni, mentre per cambi di luminosità e contrasto (così come la sfocatura), abbiamo pensato a possibili condizioni atmosferiche che possono agire sulla camera e sull'ambiente in caso di contesto reale (es. Pioggia, contesto serale e poco illuminato ma anche nebbia).
    - Generazione Mosaici: Crea 500 immagini sintetiche unendo 4 immagini casuali in una griglia 2x2 (effetto Mosaic), ricalcolando in automatico le etichette per gli oggetti all'interno del mosaico. Tale augumentation l'abbiamo scelta per il semplice motivo di andare a ridurre l'effetto della centralità spostando infatti le bounding box di diverse immagini sugli angoli delle stesse.

4. Analisi e Diagnostica: nel file dataset_audit andiamo a controllare che le fasi precedenti siano andate a buon fine e capire la composizione esatta dei dati prima di avviare l'addestramento. Conteggio e Analizza la cartella aumentata e genera un report diagnostico per i set di Train e Val, stampa poi una tabella che mostra quante istanze di ogni classe provengono dalle

immagini originali, quante dalle trasformazioni albumentations e quante dai mosaici. Conta le immagini senza oggetti (background puro), che sono utili per insegnare al modello a ridurre i falsi positivi.

5. addestamento, nel file addestramento.py, è il file che lancia fisicamente l'addestramento della rete neurale tramite la libreria ultralytics. Genera dinamicamente il file dataset.yaml richiesto da YOLO, mappando i percorsi e le 17 classi rilevate (da stop_s a parking); Invece di addestrare un singolo modello, impostiamo un ciclo per testare automaticamente diverse combinazioni di architetture (YOLO 11 Nano, YOLO11 Small, YOLO26 Nano) e diverse risoluzioni di immagine (256x256 e 320x320) per vedere con quali otteniamo risultati migliori sulla macchina. Eseguiamo l'addestramento per 150 epoche con un batch size di 64. Un dettaglio molto importante è l'impostazione augment=False: le augmentation native di YOLO sono state disabilitate di proposito, poiché i dati sono già stati potenziati dallo script augment.py.

6. Infine, il modello va convertito in .engine (tensorRT) per renderlo più veloce. Questo è un passaggio fondamentale in quanto la macchinina ha un proessore abbastanza datato e regge solo Python 3.6, il problema è che userlytics (libreria che contiene come detto precedentemente la famiglia YOLO), ha dipendenze che richiedono Python 3.8 e la conversione è stata l'unica strada percorribile, questo è un passo che eseguiamo direttamente sul terminale di NoMachine, la piattaforma software che utilizziamo per interfacciarci con la macchinina.

3 Risultati

3.1 Dataset finale

Abbiamo ri-eseguito il report sul dataset augumentato. Abbiamo ottenuto i risultati che mostriamo nelle fotografie seguenti. Come notiamo, le classi sono più proporzionate. Abbiamo ancora delle classi minoritarie ma abbiamo preferito lasciare alcune classi più rappresentate (come ad esempio i pedoni). Abbiamo quasi completamente risolto il problema del bias della centralità come mostrato dalla Figure 10.

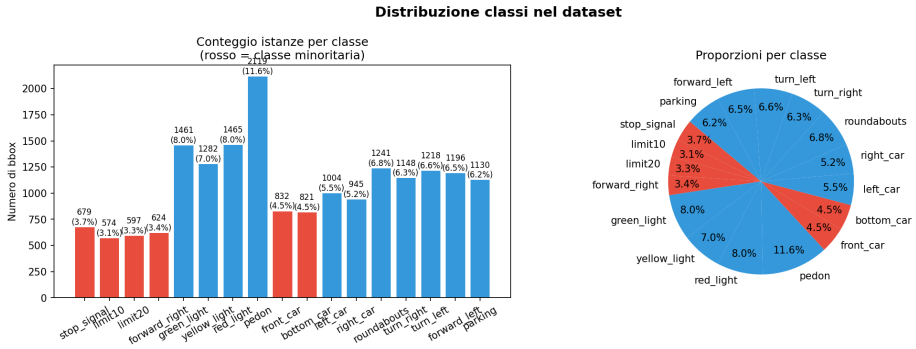


Figure 9: Analisi distribuzione delle classi dataset finale con l'aggiunta delle classi

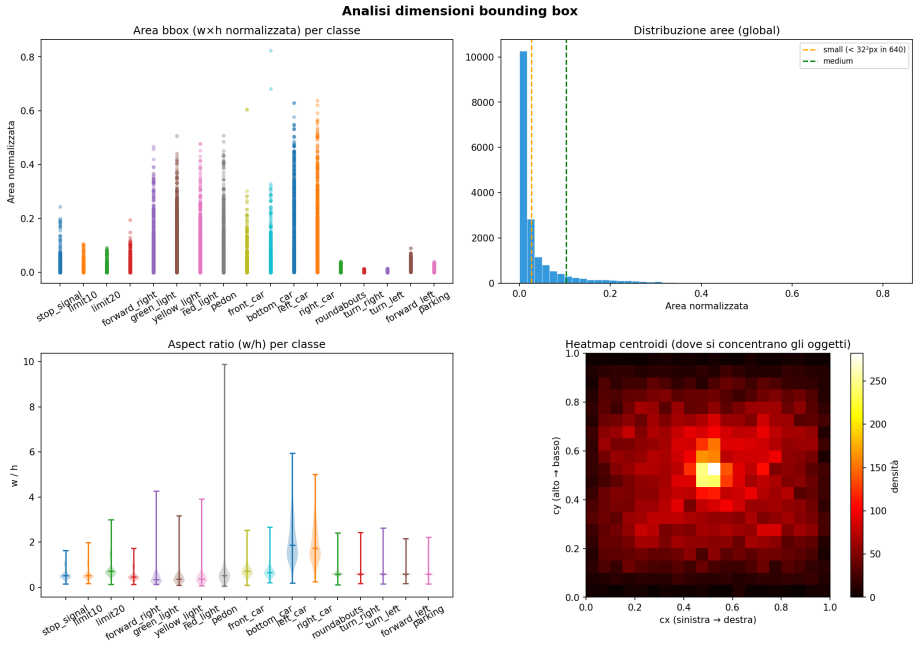


Figure 10: Analisi Bounding Box dataset finale con l'aggiunta delle classi

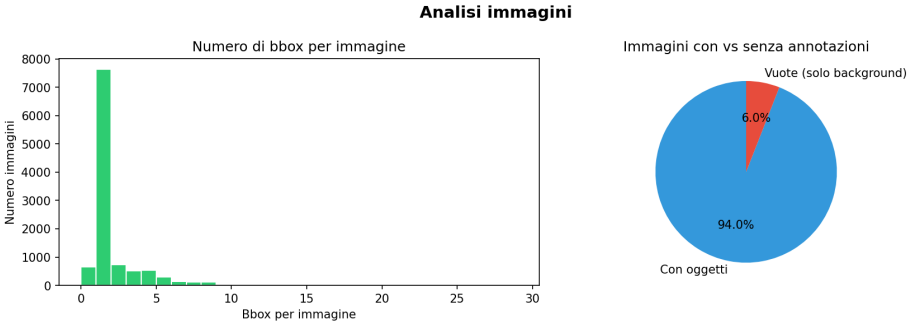


Figure 11: Analisi immagini dataset finale con l'aggiunta delle classi

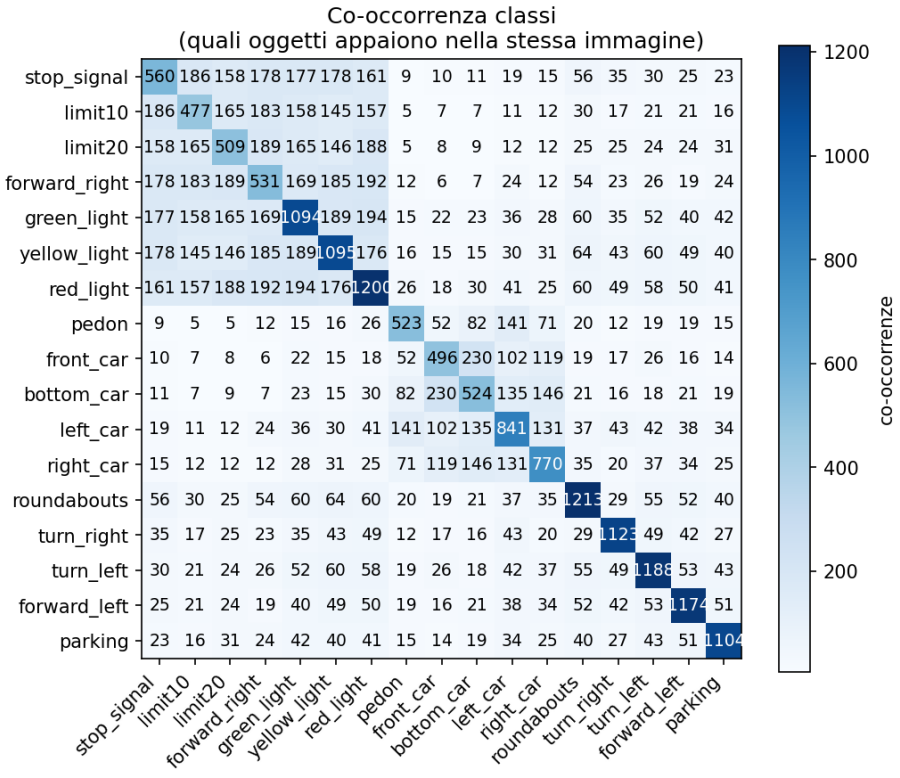


Figure 12: Co-occorrenza tra le varie classi dataset finale con l'aggiunta delle classi

3.2 Metriche da osservare

Di seguito vediamo le metriche varie che prendiamo in considerazione per valutare i nostri modelli e quale "fitta" meglio con il nostro contesto del problema, ecco le metriche e una piccola introduzione:

- Precision: Indica la percentuale di previsioni corrette rispetto a tutte le previsioni fatte. $Precision = \frac{TP}{TP+FP}$ Una bassa precisione significa che il modello genera molti falsi positivi (scambia un cartellone pubblicitario per un segnale di limite di velocità)
- Recall: Indica la percentuale di segnali stradali reali che il modello è riuscito a individuare. $Recall = \frac{TP}{TP+FN}$ Un basso recall significa che il modello genera falsi negativi (manca completamente un segnale di STOP).
- mAP50: Calcola l'Average Precision (l'area sotto la curva Precision-Recall) per ogni classe di segnale stradale considerando un rilevamento corretto se l'IoU è ≥ 0.5, e ne fa la media. È un'ottima metrica per capire se il modello "vede" i cartelli e li classifica correttamente, anche se la bounding box non è perfetta al millimetro.
- mAP50-95: È la metrica più severa e completa. Calcola il mAP a diverse soglie di IoU (dal 50% al 95% con incrementi del 5%) e ne fa la media. Un punteggio alto qui significa che il modello non solo individua il cartello, ma disegna una bounding box estremamente precisa attorno ad esso.
- FPS: Frame per second, quante immagini (fotogrammi) al secondo il modello elabora.

3.3 Ottimizzazione per la nostra auto

L'auto ha rappresentato un problema durante lo sviluppo del progetto. La Jetson Auto in combo con la Jetson Nano ci fornisce il vantaggio rispetto all'auto precedente dell'avere la GPU disponibile ma ha un processore che purtroppo non riusciva a sostenere le richieste dello script per far girare il modello sul processore. Abbiamo dunque dovuto spostare tutto sulla GPU e ottimizzato il codice. Inizialmente eravamo partiti da 2 FPS e, con l'ottimizzazione siamo arrivati tra i 25 e i 30 con i nostri modelli.

3.4 Risultati ottenuti e analisi

Come detto prima, abbiamo addestrato 3 modelli con due risoluzioni di immagini, 256x256 e per un totale di 6 modelli, ma non sono gli unici. Infatti, inizialmente, avevamo fatto partire altre run di esperimenti per l'addestramento dei vari modelli: di seguito elenchiamo tutte le metriche per i modelli (NB. i vari esperimenti precedenti sono i primi ad essere scritti in tabella, sono inseriti in ordine crescente, dal 0 a 5).

Table 1: Confronto delle prestazioni dei modelli YOLO alla loro miglior epoca

Run	Modello	Size	Precision	Recall	mAP50	mAP50-95	FPS
1	YOLO 11s	640	0.9177	0.9183	0.9498	0.7626	15
2	YOLO 11s	320	0.9023	0.8834	0.9187	0.7135	19
3	YOLO 11s	640	0.9046	0.9103	0.9367	0.7466	15
4	YOLO 11s	340	0.8235	0.8348	0.8543	0.6486	19
5	YOLO 11s	640	0.9038	0.8767	0.9134	0.7227	15
6	YOLO 11n	256	0.8177	0.8624	0.8262	0.6629	30
7	YOLO 11n	320	0.8218	0.8837	0.8479	0.6905	27
8	YOLO 11s	256	0.8272	0.8885	0.8470	0.6896	28
9	YOLO 11s	320	0.8430	0.8890	0.8656	0.7183	25
10	YOLO 26n	256	0.8055	0.8352	0.8159	0.6482	15
11	YOLO 26n	320	0.8120	0.8434	0.8361	0.6811	18
12	YOLO 11s	320	0.936	0.920	0.944	0.799	25

Le prime 5 run sono quelle che hanno ottenuto risultati più alti ma sono rimasti solo esperimenti, questo perché tutti e 5 gli esperimenti sono così proprio perché stavamo cercando dei parametri che potessero migliorare i risultati. Le run 1 e 2 erano sul dataset dell'anno scorso ma pulito, mentre le run 3 e 4 ripetono i parametri rispetto alle run 1 e 2 ma sul dataset dell'anno scorso ancora sporco. Abbiamo mostrato come si ha un aumento delle performance con la sola pulizia. Tuttavia, i modelli raggiungono performance molto alte perché molte immagini erano ripetute e finivano sia nel train che nel test ma, come abbiamo già detto, crollava a livello di prestazioni in fase di test. Dalla run 6 in poi abbiamo iniziato ad addestrare sul nuovo dataset, pulito, augumentato e con le nuove classi. Tuttavia solo dal test 6 abbiamo effettivamente iniziato a testare sul dataset definitivo. Tra tutte le run che abbiamo effettuato, la migliore è la run 9 con YOLO 11s; tuttavia, a questo punto, ci siamo resi conto di un problema: le immagini in fase di addestramento venivano confuse tra left_side_car e right_side_car e tra turn_left e turn_right. Abbiamo analizzato il modello e ci siamo resi conto che YOLO attiva in automatico fliplr, ossia "specchia" le immagini, cosa che diventa dannosa per il modello. Nella run 12 prendiamo il modello che raggiunge le migliori performance e settiamo fliplr = 0.0 e flipud = 0.0. Qui abbiamo l'intuizione che da la svolta al nostro progetto, con un modello che raggiunge performance altissime (migliori rispetto ai precedenti modelli) sotto tutti i punti di vista.

3.5 Modello migliore

Come detto, i risultati migliori li abbiamo raggiunti dalla run 12, andiamo a vedere nello specifico i risultati ottenuti:

- mAP50 (Mean Average Precision a IoU 0.5): 0.944 (94.4%); questo è un risultato fantastico. Significa che, considerando una sovrapposizione

accettabile tra il box predetto e quello reale, il modello riconosce e classifica correttamente gli oggetti in quasi il 95% dei casi.

- mAP50-95 0.799 (79.9%); Questa è una metrica molto più severa (valuta quanto i bounding box sono precisi e aderenti all'oggetto). Arrivare a questo risultato è un indicatore di altissima precisione nel posizionamento dei bounding box.
- Precision e Recall: La Precision è molto alta (93.6%), mentre la Recall è intorno all'92%. Questo indica che il modello è molto sicuro delle previsioni che fa (pochi falsi positivi), anche se potrebbe perdersi per strada qualche oggetto (qualche falso negativo in più).
- Trend delle Loss: Sia la train loss (box, cls, dfl) che la val loss stanno scendendo in modo costante. Non ci sono segni evidenti di overfitting; la loss di validazione non ha iniziato a risalire, il che significa che il modello sta ancora generalizzando bene.

Dobbiamo ovviamente fare attenzione perché la risoluzione a 320 px non è ottimale per l'addestramento ma rappresenta un buon trade-off con gli FPS raggiunti (con 640 si raggiungono tra i 15 e i 20 FPS, troppo poco per questo progetto).